

KOGNIX Enterprise AI Platform

Production-Grade Air-Gapped Docker Reference Architecture

Document Type: Solution Architecture

Deployment Model: Docker-Based On-Premises / Private Cloud

Operating Mode: Fully Air-Gapped / No Runtime Internet Access

Target Environment: Regulated Enterprise, Government, Defense, Healthcare, Finance, Critical Infrastructure

Core Products: KOGNIX AI STUDIO, KOGNIX Intelligence, KOGNIX Analytics

1. Executive Summary

The KOGNIX Enterprise AI Platform is a sovereign, air-gapped generative AI platform designed for organizations that require enterprise AI capabilities while maintaining complete control over data, models, infrastructure, networking, telemetry and operational processes.

The platform combines three core capabilities:

- **KOGNIX AI STUDIO** — application development, workflow orchestration, RAG and agent layer.
- **KOGNIX Intelligence** — local model serving and GPU-accelerated inference layer.
- **KOGNIX Analytics** — observability, tracing, evaluation, prompt management and audit layer.

The platform is deployed using **Docker containers rather than Kubernetes**. Docker Compose and native Docker services provide the application orchestration model, while dedicated physical or virtual servers provide workload isolation, GPU compute, persistent storage and high availability.

The production environment operates inside an explicit air-gapped security boundary. There is no runtime dependency on public cloud AI APIs, external model registries, public package repositories, Internet DNS, external telemetry services or SaaS infrastructure.

2. Architecture Objectives

The Docker-based architecture must provide:

1. Complete runtime air-gap.
2. Local LLM inference.
3. GPU acceleration.
4. Enterprise RAG.
5. AI application orchestration.
6. Centralized observability.
7. Multi-user and multi-project operation.
8. Persistent enterprise storage.
9. High availability.
10. Backup and disaster recovery.

11. Enterprise identity integration.
12. Network segmentation.
13. Offline software/model updates.
14. Container image provenance and signing.
15. Centralized audit logging.
16. Operational simplicity compared with Kubernetes.

The original requirements identify the same three functional layers: application/workflow, inference/model serving and observability/evaluation.

3. Product Architecture

Product	Layer	Primary Role
KOGNIX AI STUDIO	Application Layer	Applications, workflows, agents and RAG
KOGNIX Intelligence	Inference Layer	Local LLM/model serving
KOGNIX Analytics	Analytics Layer	Tracing, evaluation and governance

4. KOGNIX AI STUDIO

KOGNIX AI STUDIO is the application and orchestration layer.

Capabilities

- Low-code/no-code application development.
- Visual workflow builder.
- DAG execution.
- Agent orchestration.
- Tool invocation.
- RAG orchestration.
- Document ingestion.
- Chunking.
- Embedding coordination.
- Retrieval.
- Prompt management.
- Prompt versioning.
- Testing.
- Multi-tenant workspaces.
- Project management.
- API exposure.

The product requirements identify KOGNIX AI STUDIO as the application and workflow layer with RAG, prompt management, workspace and API capabilities.

5. KOGNIX Intelligence

KOGNIX Intelligence provides local model serving.

Capabilities

- Local LLM inference.
- Multiple open-weight models.
- GPU acceleration.
- Concurrent inference.
- Batching.
- Continuous batching where supported.
- Local model catalog.
- Model metadata.
- Model versioning.
- Embedding-model serving.
- Optional multimodal inference.
- Health monitoring.

Models may include approved open-weight models such as Llama, Mistral and Qwen.

6. KOGNIX Analytics

KOGNIX Analytics provides centralized AI observability.

Capabilities

- End-to-end request tracing.
- Workflow spans.
- LLM observations.
- Retrieval telemetry.
- Prompt management.
- Experiments.
- Token utilization.
- Latency.
- Error rates.
- Success rates.
- RAG evaluation.
- Agent evaluation.
- Audit logs.
- Engineering dashboards.
- Security/compliance dashboards.

The source architecture explicitly identifies traces, spans, observations, latency, token usage, cost, error rates, evaluation metrics and audit logging.

7. Docker-Based Architecture

Unlike the Kubernetes architecture, this deployment uses:

- Docker Engine.
- Docker Compose.
- Docker networks.
- Docker volumes.
- Docker secrets or external secret management.
- Dedicated Docker hosts.
- GPU-enabled Docker runtime.
- HAProxy/Nginx/F5 for external HA.
- Enterprise databases/storage on dedicated hosts where appropriate.

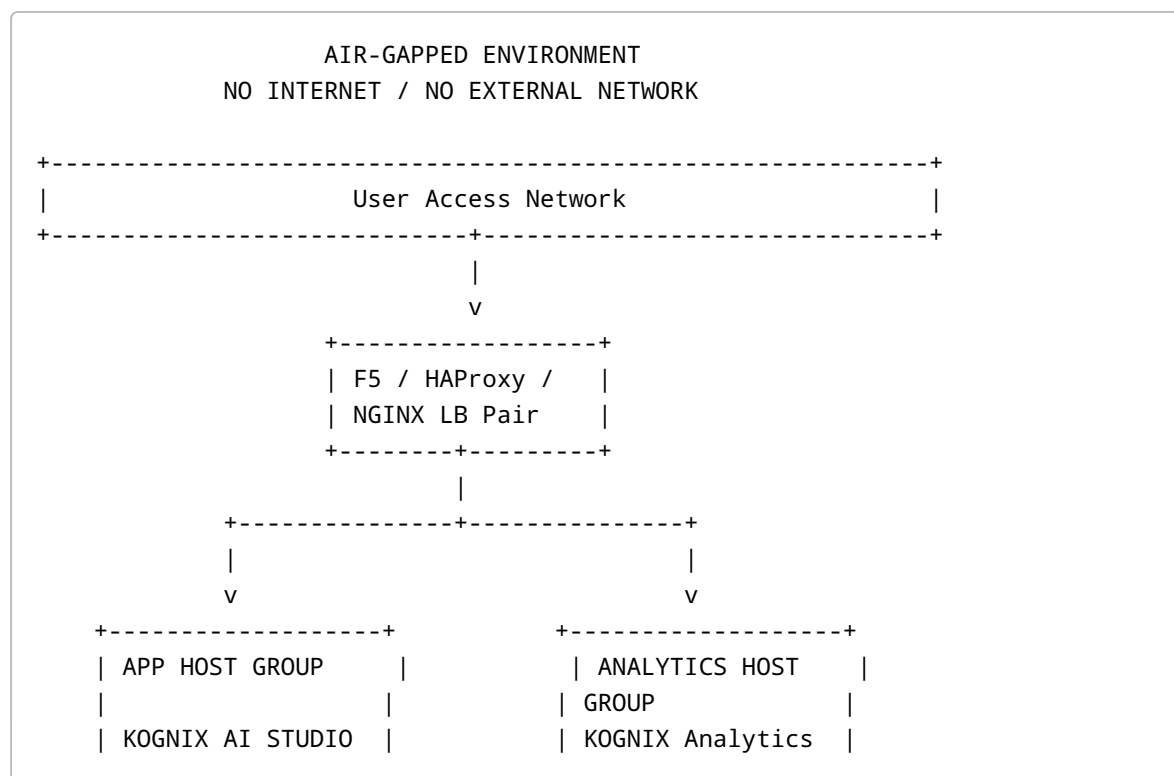
The architectural principle is:

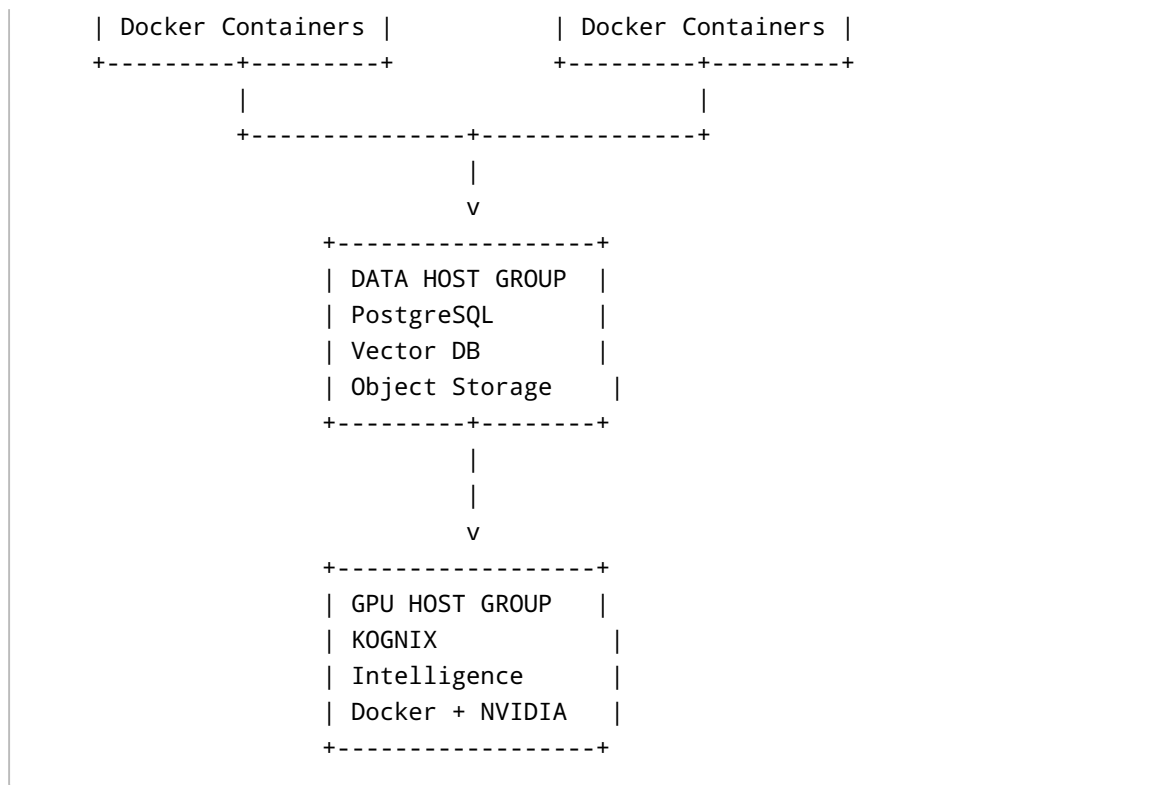
Docker provides container isolation and service packaging; the underlying enterprise infrastructure provides compute HA, storage HA, network HA and service failover.

8. Reference Physical Architecture

A production deployment should use multiple Docker hosts rather than placing the entire platform on a single server.

Recommended logical host groups:





9. Docker Host Architecture

The recommended deployment separates workloads across physical or virtual hosts.

Host Group A — Application

Runs:

- KOGNIX AI STUDIO frontend.
- KOGNIX AI STUDIO API.
- Workflow services.
- RAG services.
- Internal API gateway.

Recommended:

2-3 Docker hosts

Host Group B — KOGNIX Intelligence

Runs:

- Model-serving containers.
- Embedding containers.

- Reranking containers.
- Optional vision/speech containers.

Recommended:

2+ GPU hosts

The number of GPU hosts is workload-dependent.

Host Group C — Analytics

Runs:

- KOGNIX Analytics frontend.
- Analytics API.
- Workers.
- Trace processing.

Recommended:

2 Docker hosts

Host Group D — Database

Runs or connects to:

- PostgreSQL.
- Vector database.
- Analytics database.

For production environments, database workloads should preferably use dedicated database infrastructure rather than treating Docker containers as the database HA mechanism.

Host Group E — Object Storage

Provides:

- Documents.
- Model artifacts.
- Backups.
- Evaluation datasets.

MinIO or equivalent may be deployed as a dedicated storage service. The original architecture identifies object storage for documents, model artifacts and backups.

10. Docker Compose Architecture

The platform should be divided into multiple Compose projects rather than maintaining one enormous `docker-compose.yml`.

Recommended structure:

```
/opt/kognix/
|
+-- ai-studio/
|   +-- compose.yml
|   +-- .env
|   +-- secrets/
|
+-- intelligence/
|   +-- compose.yml
|   +-- models/
|
+-- analytics/
|   +-- compose.yml
|
+-- data/
|   +-- compose.yml
|
+-- observability/
|   +-- compose.yml
|
+-- registry/
|   +-- compose.yml
|
+-- backup/
|   +-- compose.yml
|
+-- config/
|   +-- certificates/
|   +-- policies/
|   +-- configuration/
```

This separation reduces blast radius and makes lifecycle management easier.

11. KOGNIX AI STUDIO Docker Services

A representative Compose deployment contains:

```
kognix-ai-studio
|
```

```

+-- frontend
|
+-- api
|
+-- workflow-engine
|
+-- worker
|
+-- scheduler
|
+-- rag-service
|
+-- ingestion-worker
|
+-- embedding-client
|
+-- internal-proxy

```

The exact container decomposition should follow the supported KOGNIX AI STUDIO distribution.

12. KOGNIX Intelligence Docker Services

```

kognix-intelligence
|
+-- inference-api
|
+-- model-runtime-01
|
+-- model-runtime-02
|
+-- embedding-runtime
|
+-- reranker-runtime
|
+-- model-registry-client

```

Each inference service can be mapped to a dedicated GPU.

Example:

```

GPU Host 01

+-----+
| Docker Engine |
|               |
| KOGNIX Intelligence |
|               |
+-----+

```



```

| +-----+ |
| | LLM Runtime | |
| | GPU 0,1,2,3 | |
| +-----+ |
|
| +-----+ |
| | Embedding Runtime | |
| | GPU 4 | |
| +-----+ |
+-----+

```

13. GPU Docker Runtime

GPU hosts require the NVIDIA container runtime.

The host architecture should provide:

- NVIDIA drivers.
- NVIDIA Container Toolkit.
- GPU device isolation.
- GPU health monitoring.
- Container-level GPU assignment.

Example conceptual configuration:

```

KOGNIX Intelligence Container
  |
  v
NVIDIA Container Runtime
  |
  v
Host NVIDIA Driver
  |
  v
Physical GPU

```

The GPU driver remains installed on the host while the model-serving application runs inside the container.

14. KOGNIX Analytics Docker Services

```

kognix-analytics
|
+-- web

```

```

|
+-- api
|
+-- worker
|
+-- evaluator
|
+-- event-ingestion
|
+-- scheduler

```

Persistent storage remains external to the application containers.

15. Data Layer

The data architecture should be separated from stateless Docker application containers.

```

+-----+
| Persistent Data Layer |
|                       |
| PostgreSQL            |
| Vector Database       |
| Analytics Database     |
| Object Storage        |
| Backup Repository      |
+-----+

```

The source architecture explicitly requires PostgreSQL, vector storage, analytics/event storage and object storage.

16. PostgreSQL Architecture

PostgreSQL stores:

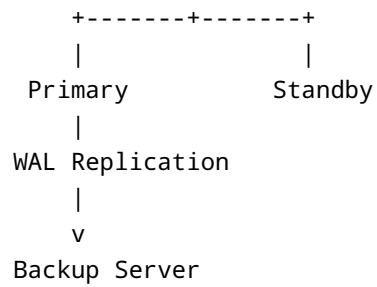
- Users.
- Projects.
- Application metadata.
- Workflow configuration.
- API keys.
- Platform configuration.

Recommended topology:

```

PostgreSQL VIP
|

```



Possible HA implementations include:

- PostgreSQL native streaming replication.
- Patroni-based HA.
- Enterprise PostgreSQL appliance.
- Managed private-cloud PostgreSQL.

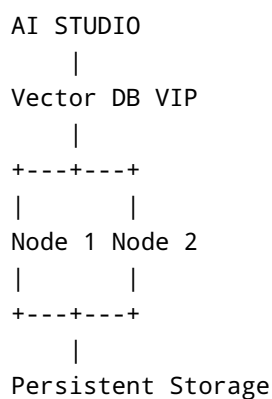
Docker should package the software, but database failover should be handled by a dedicated HA mechanism.

17. Vector Database

The vector database stores:

- Embeddings.
- RAG indexes.
- Knowledge bases.
- Retrieval metadata.

Recommended architecture:

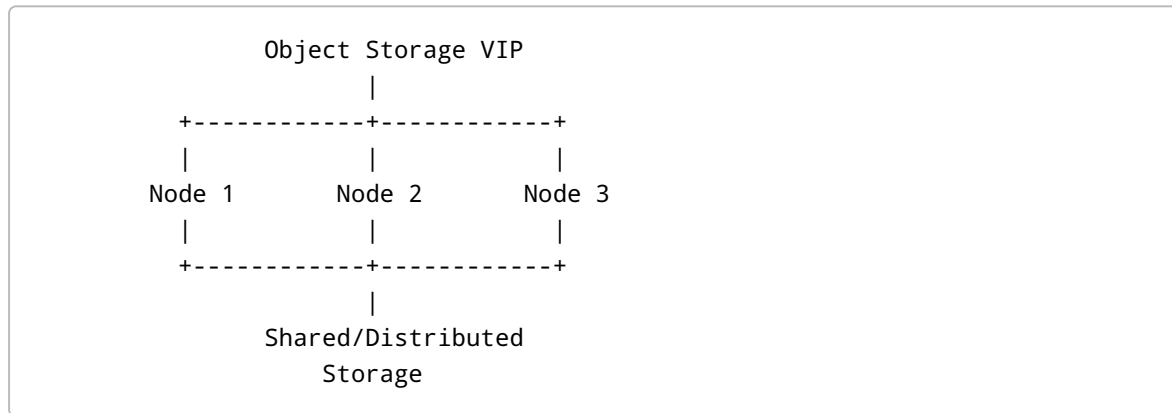


Backups must include both:

- Vector database metadata.
- Source documents from object storage.

18. Object Storage

Recommended object-storage architecture:



MinIO or an equivalent enterprise object store can provide:

- Documents.
- Model artifacts.
- Backup objects.
- Evaluation datasets.

The source architecture specifically allows MinIO or equivalent and recommends replication/erasure coding.

19. Docker Network Architecture

Docker networks should correspond to trust boundaries.

Recommended networks:

```
kognix_frontend_net
kognix_application_net
kognix_inference_net
kognix_data_net
kognix_analytics_net
kognix_management_net
```

Containers should be attached only to the networks they require.

20. Docker Network Isolation

Example:

```

AI STUDIO
|
+---- frontend_net
|
+---- application_net
|
+---- inference_net
|
+---- data_net

Intelligence
|
+---- inference_net
|
+---- analytics_net

Analytics
|
+---- analytics_net
|
+---- data_net

```

A container should not receive unrestricted access to all Docker networks.

21. External Network Isolation

Docker itself must not be treated as the primary air-gap control.

The physical firewall must enforce:

```

DEFAULT DENY

```

for outbound Internet traffic.

Even if a container is compromised, it should have no routable path to the Internet.

Recommended architecture:

```

Docker Host
|
v
Server VLAN
|
v
Internal Firewall
|

```

X
Internet

The air gap therefore exists at the network/security architecture level rather than merely at the Docker configuration level.

22. VLAN Architecture

A reference segmentation is:

VLAN	Subnet	Purpose
110	10.110.0.0/24	Users
120	10.120.0.0/24	AI STUDIO
130	10.130.0.0/24	GPU/Inference
140	10.140.0.0/24	Analytics
150	10.150.0.0/24	Data
160	10.160.0.0/24	Management
170	10.170.0.0/24	Security/SIEM
180	10.180.0.0/24	Backup
190	10.190.0.0/24	Registry

These are reference addresses and must be adapted to the customer's network plan.

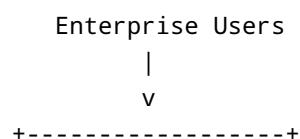
23. Load Balancing

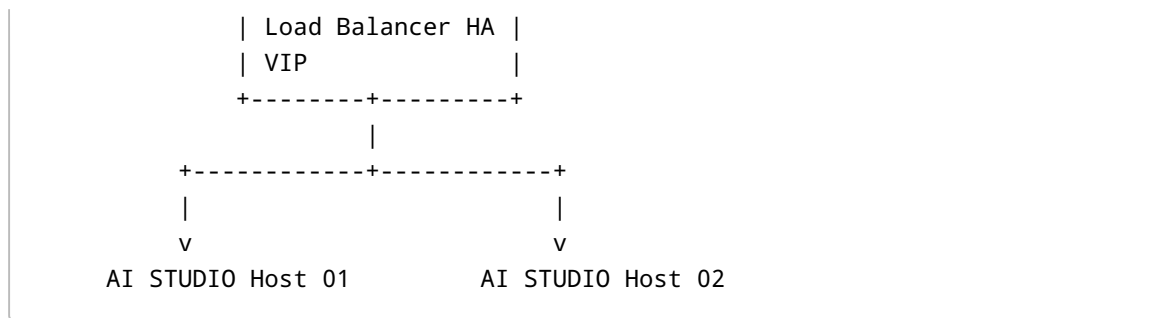
Use an enterprise load balancer in front of the Docker hosts.

Possible implementations:

- F5.
- HAProxy.
- NGINX.
- Enterprise ADC.

Architecture:





This allows application containers to remain stateless.

24. High Availability Model

Docker Compose does not itself provide the same cluster scheduler and self-healing semantics as Kubernetes.

Therefore HA must be implemented using:

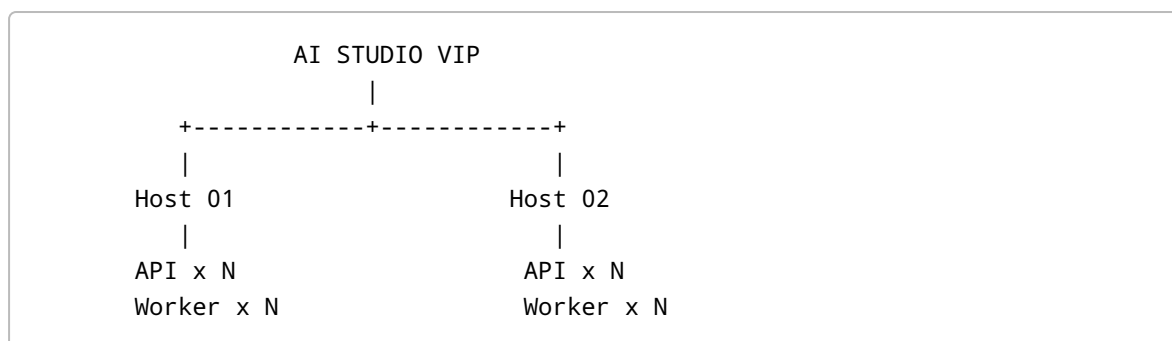
- Multiple Docker hosts.
- External load balancers.
- Service replication.
- Database HA.
- Storage HA.
- Host monitoring.
- Automated restart.
- Infrastructure-level failover.

This is a critical architectural distinction.

25. Application HA

Deploy multiple copies of stateless KOGNIX AI STUDIO services.

Example:



The load balancer performs health checks.

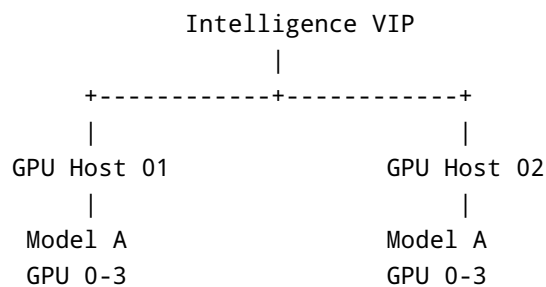
If Host 01 fails:

```
Traffic
|
X Host 01
|
v
Host 02
```

26. KOGNIX Intelligence HA

GPU workloads require more careful handling.

Recommended architecture:



A model should be deployed on multiple GPU hosts when uninterrupted availability is required.

If one GPU host fails, traffic is redirected to the surviving host.

27. Model Placement Strategy

Models should be classified into:

Tier 1 — Critical Models

Replicated across multiple GPU hosts.

Tier 2 — Standard Models

One active instance plus standby capacity.

Tier 3 — Specialized Models

Started only when required.

This prevents expensive GPU resources from being permanently consumed by low-frequency models.

28. Container Restart Policy

Stateless services should use automatic restart.

Conceptually:

```
restart: unless-stopped
```

However, restart policies do not replace:

- Health checks.
 - Load balancer failover.
 - Host-level monitoring.
 - Database HA.
 - Backup.
-

29. Docker Health Checks

Each service should expose an appropriate health endpoint.

Examples:

```
GET /health  
GET /ready  
GET /metrics
```

Health should distinguish between:

- Process alive.
 - Service ready.
 - Dependency available.
 - Model loaded.
-

30. API Gateway

An internal API gateway should provide:

- TLS termination.
- Authentication integration.
- Rate limiting.

- Request routing.
- API versioning.
- Access logging.

Example:

```

User/Application
  |
  v
API Gateway
  |
  +----> AI STUDIO
  |
  +----> Intelligence
  |
  +----> Analytics

```

Direct access to internal containers should be prohibited.

31. Service-to-Service Flow

The primary runtime path is:

```

User
  |
  HTTPS
  |
  Load Balancer
  |
  HTTPS
  |
  KOGNIX AI STUDIO
  |
  +---- HTTPS ----> KOGNIX Intelligence
  |
  +---- TCP/HTTPS -> Vector DB
  |
  +---- TCP/HTTPS -> PostgreSQL
  |
  +---- HTTPS ----> KOGNIX Analytics

```

This corresponds to the source-defined user query flow from AI STUDIO through RAG and KOGNIX Intelligence, with telemetry sent to KOGNIX Analytics.

32. Reference Port Matrix

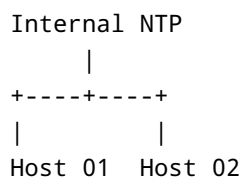
A production deployment should explicitly control ports.

Source	Destination	Port	Protocol	Purpose
Users	Load Balancer	443	HTTPS	User access
LB	AI STUDIO	443	HTTPS	Application
AI STUDIO	Intelligence	443	HTTPS	LLM API
AI STUDIO	PostgreSQL	5432	TCP	Metadata
AI STUDIO	Vector DB	Vendor-specific	TCP/HTTPS	Retrieval
AI STUDIO	Analytics	443	HTTPS	Telemetry
Analytics	PostgreSQL	5432	TCP	Metadata
Analytics	Analytics DB	Vendor-specific	TCP	Events
Intelligence	Registry	443	HTTPS	Model artifacts
Hosts	Internal DNS	53	DNS	Name resolution
Hosts	NTP	123	UDP	Time synchronization
Platform	SIEM	Approved	TCP/UDP/HTTPS	Logging
Backup	Data	Approved	TCP	Backup

Only ports explicitly required by the deployed software should be enabled.

33. Time Synchronization

All hosts must use a controlled internal time source.



Accurate time is essential for:

- Audit logs.
- Security investigations.
- Trace correlation.
- Certificate validation.
- Incident response.

No external NTP should be required at runtime.

34. Identity & Access Management

KOGNIX should integrate with internal:

- Active Directory.
- LDAP.
- SAML.
- OIDC.

The source requirements explicitly identify internal IdP integration and RBAC/ABAC.

Recommended access hierarchy:

```
Enterprise Identity
  |
  v
KOGNIX
  |
  +-- Tenant
  |
  +-- Workspace
  |
  +-- Project
  |
  +-- Application
  |
  +-- Workflow
```

35. Container Secrets

Secrets should never be stored directly in:

- Dockerfiles.
- Git repositories.
- Compose files.
- Container images.

Use:

- Docker secrets where appropriate.
- Enterprise vault.
- HSM/KMS-backed secrets.
- Protected configuration files.

36. TLS Architecture

TLS should be used for:

- User → Load Balancer.
- Load Balancer → AI STUDIO.
- AI STUDIO → Intelligence.
- AI STUDIO → Analytics.
- AI STUDIO → databases where supported.
- Registry communication.
- Object storage.
- Administrative interfaces.

The source architecture explicitly requires TLS for internal service-to-service communication.

37. Encryption at Rest

Encrypt:

- PostgreSQL.
- Vector database.
- Analytics storage.
- Object storage.
- Model files.
- Backup files.
- Audit logs.

Where available, use customer-managed keys through an enterprise KMS/HSM.

38. Container Image Security

Before an image enters the air-gapped environment:

1. Download through approved external process.
2. Generate checksum.
3. Scan for vulnerabilities.
4. Scan for malware.
5. Inspect SBOM.
6. Validate licenses.
7. Sign image.
8. Transfer offline.
9. Verify signature.
10. Import into internal registry.

The source requirements explicitly call for signed container images and vulnerability scanning before importing them into the air-gapped environment.

39. Internal Container Registry

Deploy an internal registry such as:

- Harbor.
- Nexus.
- Enterprise registry.

Architecture:

```
Offline Update Station
  |
  v
Signed Container Image
  |
Offline Transfer
  |
  v
Internal Registry
  |
+----> AI STUDIO Hosts
  |
+----> Intelligence Hosts
  |
+----> Analytics Hosts
```

The original architecture specifies an internal Harbor/Nexus-style repository.

40. Internal Model Registry

Model artifacts should be managed independently from container images.

Example:

```
Model Registry
|
+-- Llama
|   +-- Model A
|
+-- Qwen
|   +-- Model B
|
```

```
+-- Mistral
|   +-- Model C
|
+-- Embeddings
|   +-- Embedding Model A
|
+-- Rerankers
    +-- Reranker A
```

Each artifact must have provenance and checksum metadata.

41. Air-Gapped Update Architecture

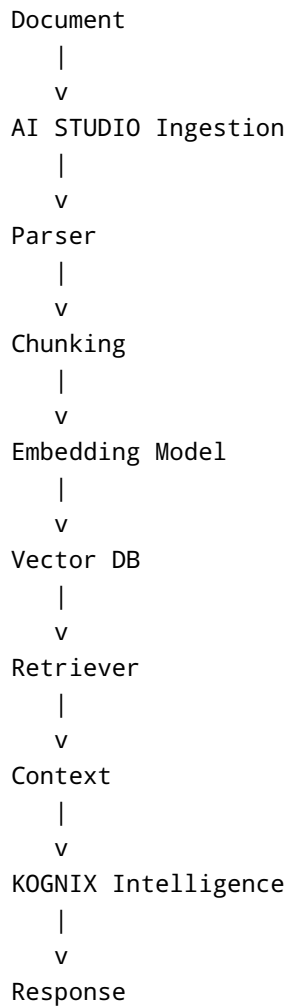
The update process is:

```
External Source
  |
  v
Hardened Update Station
  |
  +-- Malware Scan
  +-- Vulnerability Scan
  +-- Signature Validation
  +-- Checksum
  +-- License Review
  |
  v
Offline Transfer
  |
  v
Air-Gapped Import Zone
  |
  v
Internal Registry
  |
  v
Staging
  |
  v
Validation
  |
  v
Production
```

This preserves the source requirement for an offline signed transfer mechanism.

42. RAG Architecture

The RAG pipeline is:



43. RAG Data Security

Access to knowledge bases should be controlled at:

- Tenant level.
- Workspace level.
- Project level.
- Application level.
- Document level where supported.

A user must not retrieve documents outside their authorization boundary.

44. KOGNIX Analytics Architecture

All major services emit telemetry:

```
AI STUDIO -----  
  
Intelligence -----+-----> KOGNIX Analytics  
                      /  
Vector/RAG -----/
```

Analytics stores:

- Trace ID.
- User ID.
- Application ID.
- Workflow ID.
- Model.
- Model version.
- Prompt.
- Retrieval information.
- Latency.
- Token counts.
- Errors.
- Evaluation results.

45. Audit Architecture

Audit logs should capture:

- User authentication.
- Authorization changes.
- Application changes.
- Workflow changes.
- Model deployment.
- Model version.
- Prompt changes.
- Data ingestion.
- Retrieval events.
- AI requests.
- Administrative changes.

The source requirements explicitly call for timestamps and user IDs for auditability.

46. SIEM Integration

The architecture supports forwarding logs to the internal SIEM.

```
graph TD
    A[Docker Hosts] --> B[Central Log Collector]
    B --> C[Internal SIEM]
```

For higher-security environments:

```
graph TD
    A[KOGNIX] --> B[One-Way Logging Gateway]
    B --> C[SIEM]
```

No return path should be required for basic log forwarding.

47. Monitoring Architecture

Monitoring should cover:

Docker

- Container status.
- Restart count.
- CPU.
- Memory.
- Disk.
- Network.

Hosts

- CPU.
- Memory.
- Filesystem.
- RAID.
- NIC.
- Temperature.
- Hardware health.

GPU

- Utilization.
- Memory.
- Temperature.
- ECC.
- Power.
- GPU availability.

Application

- API latency.
- Requests/sec.
- Errors.
- Queue depth.

AI

- Tokens/sec.
- Prompt tokens.
- Completion tokens.
- Model latency.
- Context size.

48. Logging Architecture

A central logging service should aggregate:

```
AI STUDIO
Intelligence
Analytics
Docker Engine
Linux
Firewall
Load Balancer
Storage
Database
    |
    v
Central Log Collector
    |
    +----> Local Log Store
    |
    +----> SIEM
```

49. Backup Strategy

Backups should cover:

Application

- Workflow definitions.
- Configuration.
- Metadata.

PostgreSQL

- Full backups.
- Incremental/WAL backups.

Vector DB

- Indexes.
- Metadata.

Object Storage

- Documents.
- Model artifacts.
- Evaluation data.

Analytics

- Metadata.
- Required historical telemetry.

Infrastructure

- Compose files.
- Configuration.
- Certificates.
- Registry metadata.

50. Backup Architecture

```
Production
|
+---- PostgreSQL
+---- Vector DB
+---- Object Storage
+---- Analytics
|
v
Backup Server
```

```
|
v
Offline Backup
|
+---- Disk
+---- Tape
+---- Secondary Air-Gapped Site
```

The source requirements explicitly include local backup, offline replication/tape and optional secondary air-gapped DR.

51. Disaster Recovery

A secondary air-gapped site is recommended for critical installations.

```
Primary AI Site
|
Offline Replication
|
v
Secondary AI Site
```

Replicate:

- PostgreSQL backups.
- Object storage.
- Model registry.
- Container registry.
- KOGNIX configuration.
- Application configuration.

52. Docker Host Hardening

Docker hosts should be hardened according to enterprise security standards.

Recommended controls:

- Minimal OS.
- No unnecessary packages.
- SSH restricted to management network.
- Root login disabled.
- MFA through privileged-access infrastructure.
- Disk encryption.
- Host firewall.
- SELinux/AppArmor where compatible.

- Audit daemon.
 - Central logging.
 - Automatic security monitoring.
-

53. Docker Daemon Security

The Docker API socket should never be exposed externally.

Avoid:

```
tcp://0.0.0.0:2375
```

Administrative access should occur through controlled host management.

The Docker socket should not be mounted into application containers unless absolutely required.

54. Container Privilege Model

Application containers should run:

- Non-root where supported.
- With minimal Linux capabilities.
- Without privileged mode.
- With read-only filesystems where possible.
- With explicit volume mounts.

GPU containers will require appropriate GPU runtime permissions but should still operate with the minimum privileges required.

55. Storage Design

The storage architecture should distinguish:

Ephemeral

Container writable layers.

Persistent

- Database data.
- Object storage.
- Vector data.
- Model artifacts.

No critical enterprise data should reside only in a container's writable layer.

56. Docker Volume Strategy

Example:

```
/opt/kognix/data/  
|  
+-- postgres/  
+-- vector/  
+-- analytics/  
+-- object-storage/  
+-- models/  
+-- backups/
```

Production storage should ideally reside on enterprise RAID/SAN/NAS/distributed storage rather than local single-disk Docker volumes.

57. Security Zones

The final architecture should contain:

```
ZONE 1 – USER  
ZONE 2 – APPLICATION  
ZONE 3 – INFERENCE  
ZONE 4 – ANALYTICS  
ZONE 5 – DATA  
ZONE 6 – MANAGEMENT  
ZONE 7 – SECURITY  
ZONE 8 – BACKUP  
ZONE 9 – REGISTRY
```

Each zone should have explicit firewall policy.

58. Reference Firewall Policy

User → Application

ALLOW

HTTPS 443.

Application → Intelligence

ALLOW

Approved inference API port.

Application → Data

ALLOW

Only required database/vector/object-storage ports.

Application → Analytics

ALLOW

Telemetry API.

Intelligence → Registry

ALLOW

HTTPS 443.

Analytics → SIEM

ALLOW

Approved logging protocol.

Management → Docker Hosts

ALLOW

Only approved management protocols.

Any Zone → Internet

DENY

User → Database

DENY

User → GPU Hosts

DENY

User → Registry

DENY

Intelligence → User Network

DENY, except through approved inference/API path.

59. Example Docker Deployment Lifecycle

1. Import approved image
- |
2. Verify signature
- |
3. Load configuration
- |
4. Load secrets
- |
5. Create Docker networks
- |
6. Mount persistent storage
- |
7. Start dependencies
- |
8. Start application services
- |
9. Run health checks
- |
10. Register service with LB
- |
11. Enable production traffic

60. Deployment Order

Recommended order:

Phase 1

Infrastructure:

- Servers.
- Networking.
- Storage.
- Firewall.
- DNS.

- NTP.

Phase 2

Security:

- Identity.
- Certificates.
- KMS.
- SIEM.
- Logging.

Phase 3

Platform:

- Docker Engine.
- NVIDIA runtime.
- Internal registry.
- Backup.

Phase 4

Data:

- PostgreSQL.
- Vector DB.
- Object storage.
- Analytics storage.

Phase 5

KOGNIX:

- AI STUDIO.
- Intelligence.
- Analytics.

Phase 6

Validation:

- Functional.
 - Security.
 - Performance.
 - HA.
 - Backup/restore.
 - Air-gap validation.
-

61. Example Use Case — Classified Document Q&A

Users

Defense/intelligence analysts.

Data

- Classified reports.
- Internal intelligence documents.
- Manuals.
- Historical records.

Flow

```
Analyst
|
AI STUDIO
|
RAG
|
Vector DB
|
KOGNIX Intelligence
|
Answer
|
KOGNIX Analytics
```

Air-Gap Requirement

The information must remain inside the classified environment.

62. Example Use Case — Government Policy Assistant

Users

Government officers and policy analysts.

Data

- Internal policies.
- Circulars.

- Procedures.
- Regulations.

Platform

AI STUDIO builds the application.

Intelligence provides local inference.

Analytics records traceability.

Air-Gap Requirement

Government information remains inside the government-controlled network.

63. Example Use Case — Secure Clinical AI

Users

Doctors and authorized clinical staff.

Data

- Clinical guidelines.
- Hospital policies.
- Approved medical knowledge.
- Patient information where authorized.

Platform

AI STUDIO provides workflow and RAG.

Intelligence provides local inference.

Analytics provides traceability.

Air-Gap Requirement

Sensitive health information does not need to leave the hospital infrastructure.

64. Example Use Case — Critical Infrastructure

Users

Engineers and operations teams.

Data

- Engineering manuals.
- Maintenance records.
- SOPs.
- Equipment documentation.

Platform

RAG knowledge assistant running entirely locally.

Air-Gap Requirement

Operational and engineering information remains within the infrastructure operator's security boundary.

65. Docker vs Kubernetes Architectural Trade-Off

Area	Docker Architecture	Kubernetes Architecture
Deployment complexity	Lower	Higher
Initial setup	Faster	More involved
Container orchestration	Compose	Kubernetes
Auto-scheduling	Limited	Advanced
Self-healing	Basic	Advanced
Horizontal scaling	Manual/external	Native
GPU scheduling	Host-level	Cluster-level
HA	Infrastructure-based	Platform-based
Operational overhead	Lower	Higher
Small/medium deployment	Strong fit	Potentially excessive
Large enterprise cluster	Possible	Stronger
Air-gapped deployment	Excellent	Excellent
Skill requirements	Docker/Linux	Kubernetes/platform engineering

For organizations that prioritize operational simplicity and have a relatively bounded AI platform footprint, Docker is a viable architecture.

For very large multi-tenant environments with dynamic workload scheduling, Kubernetes becomes more attractive.

66. Docker Architecture — Key Design Constraint

The most important architectural difference is that **Docker Compose is not a complete distributed-cluster management platform**.

Therefore the production architecture must not rely on:

```
docker compose
```

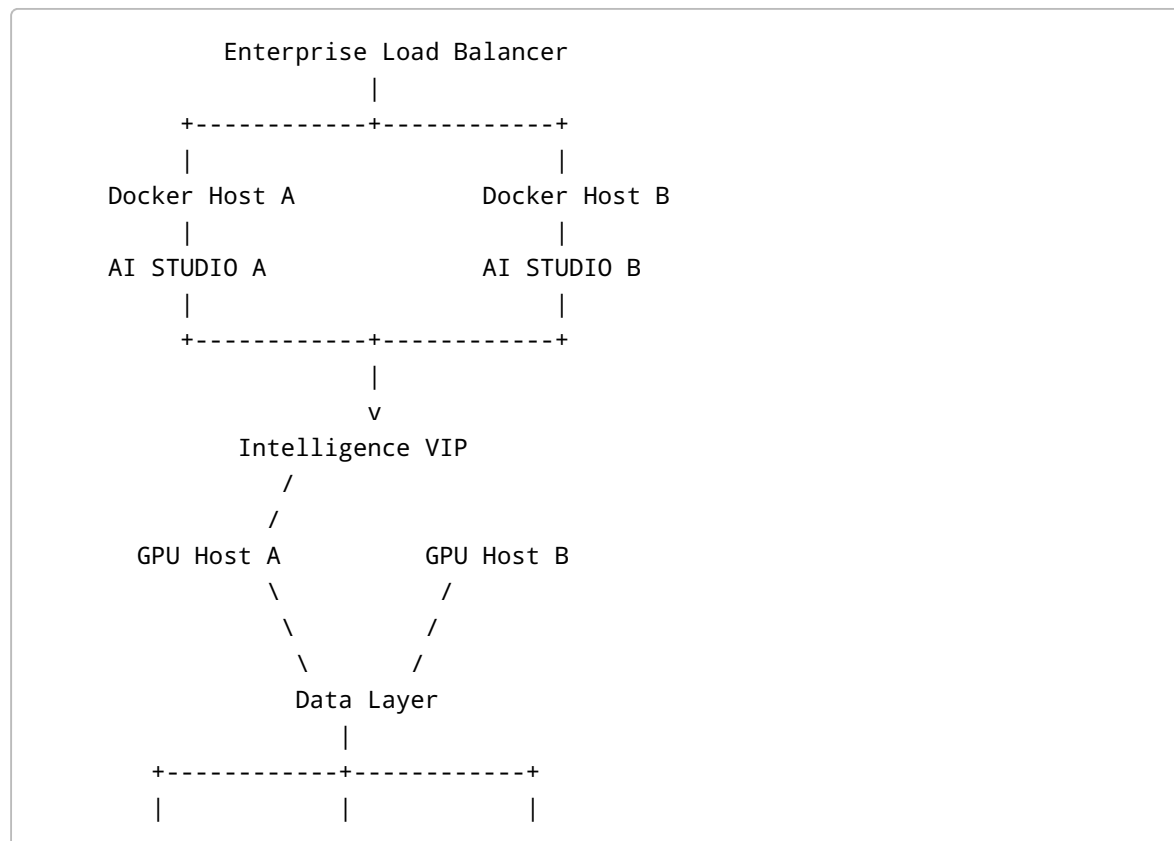
alone for:

- Automatic cross-host failover.
- Distributed scheduling.
- Database HA.
- Storage HA.
- GPU scheduling.
- Disaster recovery.

Those capabilities must be provided by the underlying infrastructure or dedicated HA products.

67. Recommended Production Pattern

The preferred enterprise pattern is therefore:



```
PostgreSQL    Vector DB    Object Store
  |
  v
Backup / DR
```

This provides the simplicity of Docker while retaining enterprise-grade infrastructure HA.

68. Air-Gap Validation

The deployment should undergo an explicit air-gap test.

Test 1

Attempt outbound Internet connectivity from an AI STUDIO container.

Expected: Failure.

Test 2

Attempt outbound connectivity from Intelligence.

Expected: Failure.

Test 3

Attempt outbound connectivity from Analytics.

Expected: Failure.

Test 4

Disable external DNS.

Expected: Platform remains operational.

Test 5

Disable Internet routing.

Expected: AI workloads remain operational.

Test 6

Disconnect the update station.

Expected: Production continues operating normally.

69. Security Acceptance Tests

Validate:

- Unauthorized user cannot access applications.
 - User cannot access another tenant.
 - User cannot access database directly.
 - User cannot access GPU hosts.
 - Containers cannot access Internet.
 - Unsigned image cannot be deployed.
 - Unauthorized model cannot be loaded.
 - Audit events are generated.
 - SIEM receives security events.
 - Backup can be restored.
-

70. High Availability Acceptance Tests

Test:

1. AI STUDIO host failure.
2. AI STUDIO container failure.
3. GPU host failure.
4. PostgreSQL primary failure.
5. Vector database node failure.
6. Analytics node failure.
7. Storage node failure.
8. Load balancer failure.

The expected outcome should be continued service or controlled recovery according to the defined SLA.

71. Operational Responsibilities

Platform Team

- Docker hosts.
- OS.
- Network.
- Storage.
- Load balancers.
- Container runtime.

AI Engineering

- Models.
- RAG.
- Prompt engineering.

- Evaluation.
- AI applications.

Security

- IAM.
- Firewall.
- Vulnerability management.
- Image signing.
- SIEM.

Compliance

- Audit.
- Retention.
- Validation.
- Regulatory controls.

Operations

- Monitoring.
- Backup.
- DR.
- Incident response.

72. Implementation Timeline

A representative implementation remains approximately **6–10 weeks after hardware readiness**, depending on:

- Hardware availability.
- Network/security approvals.
- Identity integration.
- Storage readiness.
- Model approval.
- Application complexity.
- Compliance validation.

The source requirements identify the same 6–10 week deployment target.

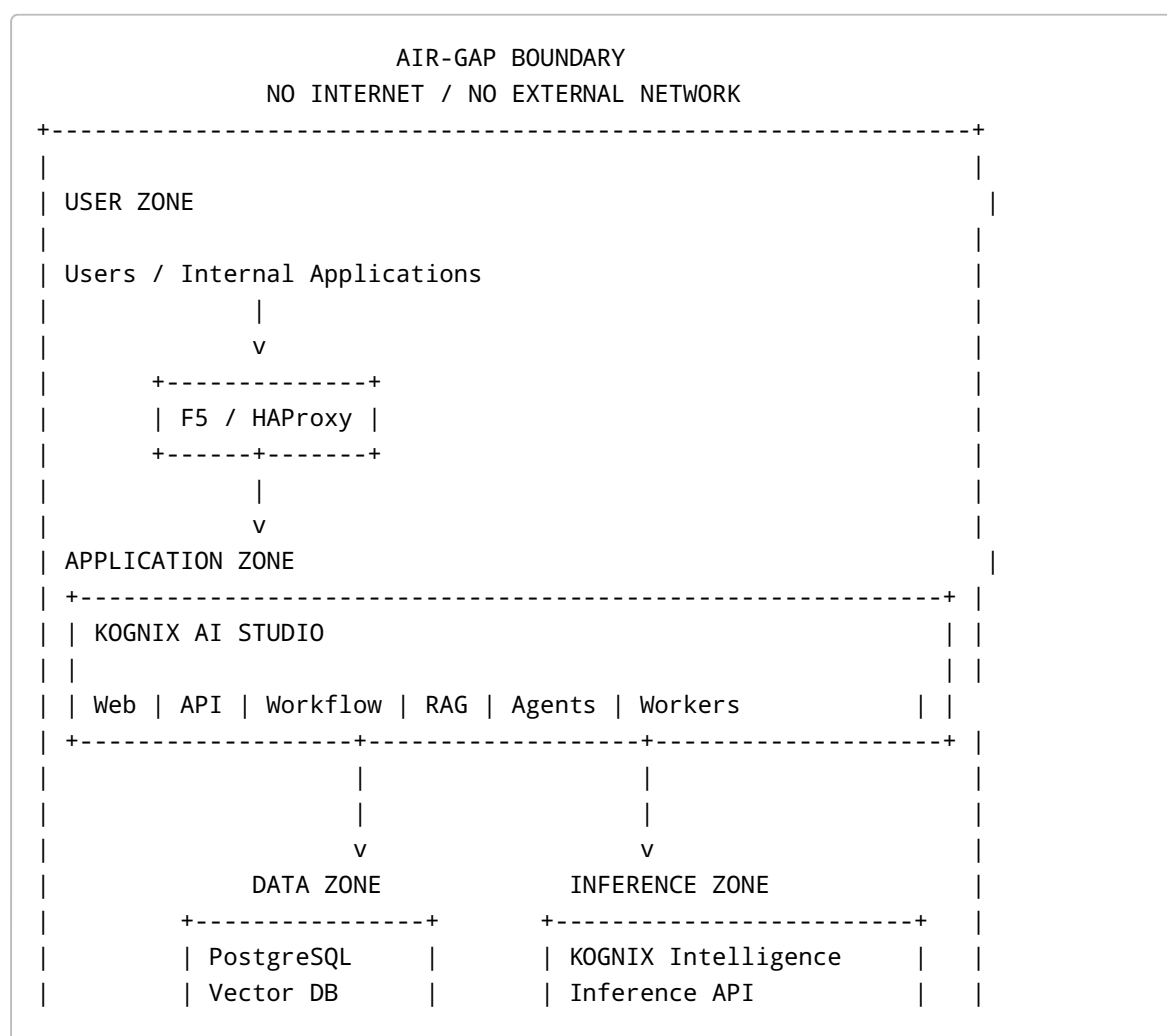
73. Recommended Docker Host Inventory

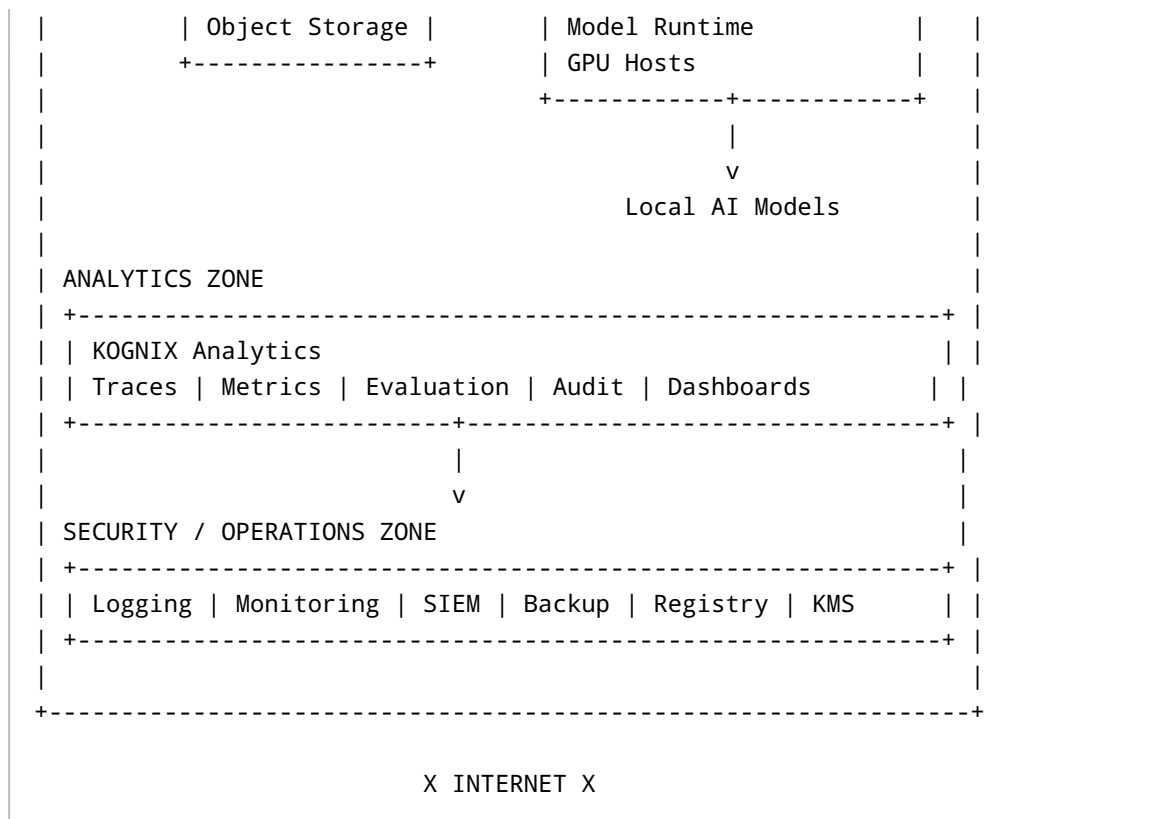
A representative medium-to-large deployment could begin with:

Host Group	Quantity	Purpose
Load Balancer	2	HA ingress
AI STUDIO	2-3	Application
Analytics	2	Analytics
GPU	2+	Inference
Database	2-3	PostgreSQL/Vector
Object Storage	3+	Documents/models
Registry	2	Internal artifacts
Monitoring	2	Operations
Backup	1-2	Backup/DR

These are reference topology numbers rather than fixed sizing requirements.

74. Reference Logical Architecture





75. Final Architecture Position

The Docker-based KOGNIX architecture provides a production-oriented alternative to the Kubernetes reference design while retaining the three-layer product architecture:

KOGNIX AI STUDIO

Build, orchestrate and expose enterprise AI applications.

KOGNIX Intelligence

Serve approved AI models locally using enterprise GPU infrastructure.

KOGNIX Analytics

Observe, evaluate, trace and govern AI workloads.

The deployment is enclosed within a strict air-gapped security boundary, with no runtime dependency on external AI providers, public model registries or external telemetry endpoints.

The critical architectural principle is that **Docker provides containerization and service packaging, while enterprise infrastructure provides HA, network isolation, persistent storage, load balancing, database failover and disaster recovery.**

This makes the architecture particularly appropriate where the customer wants a simpler operational footprint than Kubernetes while still requiring enterprise-grade AI infrastructure.

The complete runtime path remains:

Enterprise User → KOGNIX AI STUDIO → RAG/Data → KOGNIX Intelligence → Local Model → KOGNIX AI STUDIO → User

with:

KOGNIX AI STUDIO + KOGNIX Intelligence → KOGNIX Analytics

for centralized observability, evaluation and audit.

The architecture therefore delivers a sovereign, locally controlled AI platform in which **data, models, inference, applications, telemetry and audit records remain inside the customer's security boundary.**